

 Cập nhật tháng 8 năm 2024

[Bài đọc] Quy tắc đặt tên cho tài nguyên phức tạp

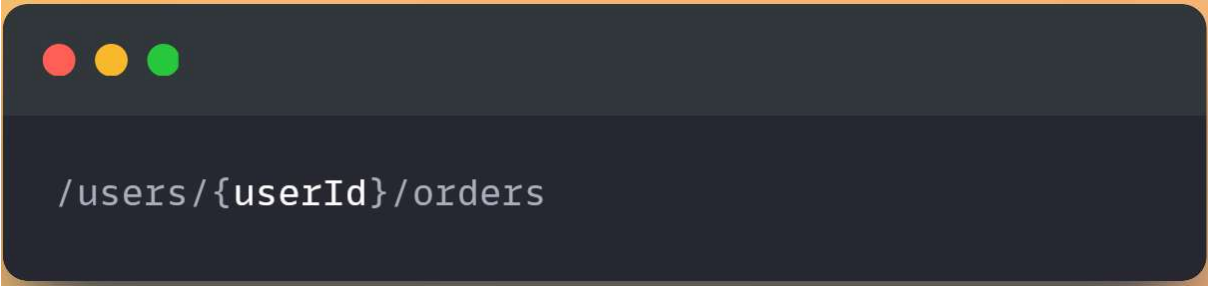
Trong phát triển API, đặc biệt là RESTful API, việc đặt tên tài nguyên đóng vai trò vô cùng quan trọng. Với các tài nguyên đơn giản, quy tắc đặt tên khá dễ áp dụng. Tuy nhiên, khi làm việc với các tài nguyên phức tạp – chẳng hạn có quan hệ cha – con, nhiều – nhiều, hoặc yêu cầu thao tác ngoài CRUD – chúng ta cần những nguyên tắc rõ ràng để đảm bảo API dễ hiểu, dễ sử dụng và dễ bảo trì

1. Tài nguyên phức tạp và sự cần thiết của quy tắc đặt tên

- Các tài nguyên phức tạp thường gắn với nhiều quan hệ dữ liệu khác nhau, đòi hỏi cách đặt tên cụ thể, đôi khi dài hơn để tránh trùng lặp và nhầm lẫn. Ví dụ: khi truy xuất bình luận của một người dùng, URL cần thể hiện rõ mối liên hệ cha – con, thay vì chỉ dừng ở cấp độ bình luận chung. Việc áp dụng quy tắc chặt chẽ sẽ giúp API thống nhất và dễ mở rộng về sau

2. Cấu trúc URL cha – con phản ánh quan hệ dữ liệu

- Một trong những nguyên tắc quan trọng nhất là thiết kế URL theo mô hình cha – con. Điều này phản ánh trực tiếp quan hệ trong cơ sở dữ liệu và làm rõ ngữ cảnh của dữ liệu cần truy xuất. Ví dụ:



```
/users/{userId}/orders
```

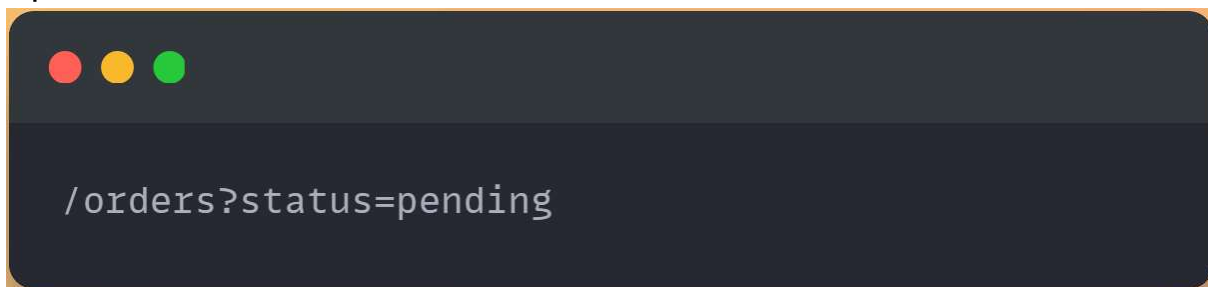
- Endpoint trên thể hiện rõ “đơn hàng của một người dùng cụ thể”. Cách đặt này không chỉ trực quan mà còn phù hợp với chuẩn RESTful, giúp việc truy vấn tài nguyên liên quan trở nên dễ dàng và hiệu quả hơn

3. Xử lý nhiều mối quan hệ trong API

- Trong thực tế, quan hệ một – nhiều (one-to-many) và nhiều – nhiều (many-to-many) xuất hiện rất phổ biến. Để thể hiện đúng các quan hệ này, URL phải được thiết kế hợp lý
 - **One-to-many:** Một người dùng có thể có nhiều bài viết
→ `/users/{userId}/posts`
 - **Many-to-many:** Một sản phẩm có thể thuộc nhiều đơn hàng, và một đơn hàng chứa nhiều sản phẩm
→ `/orders/{orderId}/products` hoặc
`/products/{productId}/orders`
- Cách đặt tên này giúp API linh hoạt hơn, cho phép lấy dữ liệu từ nhiều hướng khác nhau mà không gây nhầm lẫn

4. Thể hiện trạng thái tài nguyên qua tham số truyền

- Nhiều tài nguyên có thể tồn tại ở các trạng thái khác nhau. Ví dụ, một đơn hàng có thể là *pending*, *completed* hoặc *canceled*. Thay vì tạo nhiều endpoint riêng biệt, chúng ta có thể truyền trạng thái qua tham số URL:

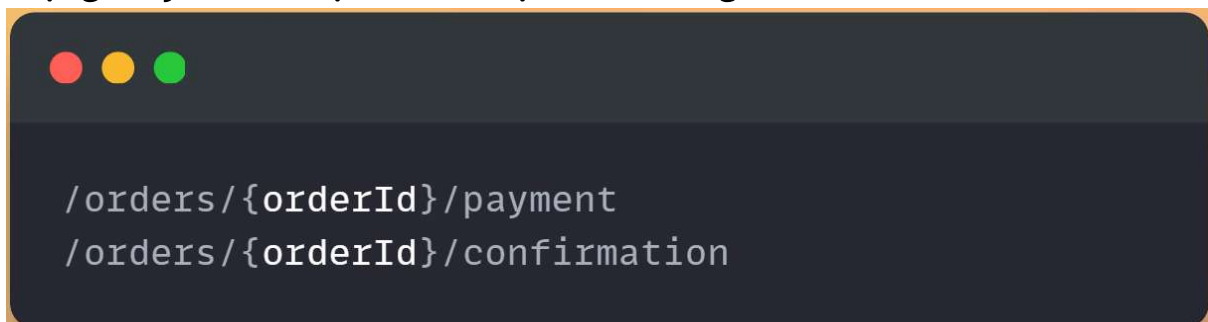


```
/orders?status=pending
```

- Cách tiếp cận này vừa gọn gàng, vừa giúp API linh hoạt trong việc lọc dữ liệu, đồng thời giảm thiểu sự trùng lặp không cần thiết

5. Hoạt động đặc thù ngoài CRUD

- Không phải lúc nào API cũng chỉ dừng lại ở Create – Read – Update – Delete. Trong nhiều trường hợp, chúng ta cần những hành động đặc thù hơn, như thanh toán, xác nhận hoặc hủy đơn hàng. Những hành động này nên được thể hiện rõ trong URL:



```
/orders/{orderId}/payment  
/orders/{orderId}/confirmation
```

- Điều này giúp API rõ ràng, trực quan và tuân thủ nguyên tắc RESTful

6. Nguyên tắc đặt tên tài nguyên dễ hiểu, thống nhất

- Một API tốt phải đảm bảo tính **nhất quán** và **dễ hiểu** trong cách đặt tên. Người dùng API chỉ cần nhìn vào URL cũng có thể đoán được chức

năng của nó. Quy tắc cơ bản thường là:

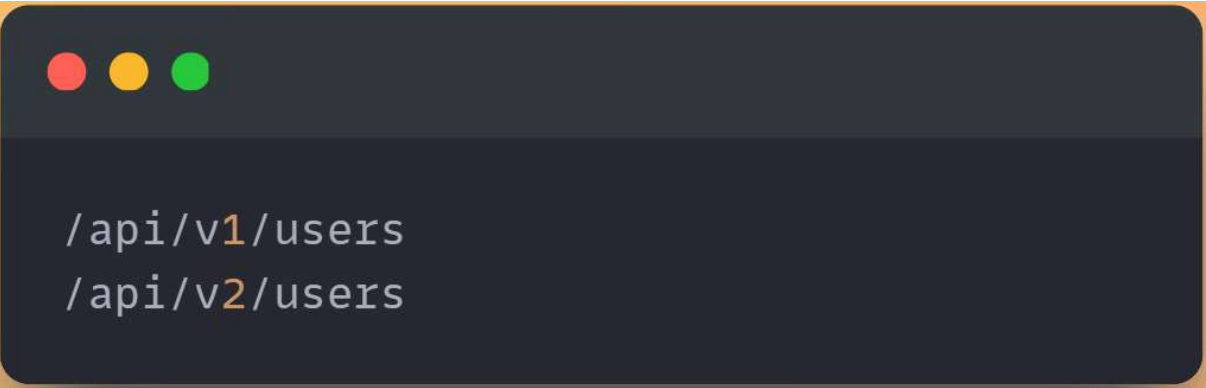


```
/{root-tài-nguyên}/{id}/{tài-nguyên-con}?{tham-số-lọc}
```

- Sự nhất quán này giúp đội ngũ phát triển dễ bảo trì và người dùng API nhanh chóng nắm bắt được cấu trúc

7. Đặt tên phiên bản API để quản lý nâng cấp

- Trong quá trình phát triển, API thường được nâng cấp để đáp ứng yêu cầu mới. Việc gắn phiên bản vào URL giúp phân biệt các phiên bản, đồng thời tránh ảnh hưởng đến những hệ thống đang dùng phiên bản cũ:



```
/api/v1/users  
/api/v2/users
```

- Đây là bước quan trọng để đảm bảo sự ổn định và phát triển lâu dài của API

8. Kết luận

- Việc đặt tên cho tài nguyên phức tạp trong API không chỉ là vấn đề cú pháp, mà còn là nghệ thuật thiết kế hệ thống. Một API với cách đặt tên rõ ràng, thống nhất và hợp lý sẽ mang lại trải nghiệm tốt cho cả người dùng lẫn nhà phát triển. Các quy tắc về cấu trúc cha - con, xử lý quan hệ nhiều chiều, sử dụng tham số cho trạng thái, bổ sung endpoint cho hành động đặc thù, và quản lý phiên bản API là những nền tảng giúp API trở nên chuyên nghiệp, dễ mở rộng và dễ bảo trì